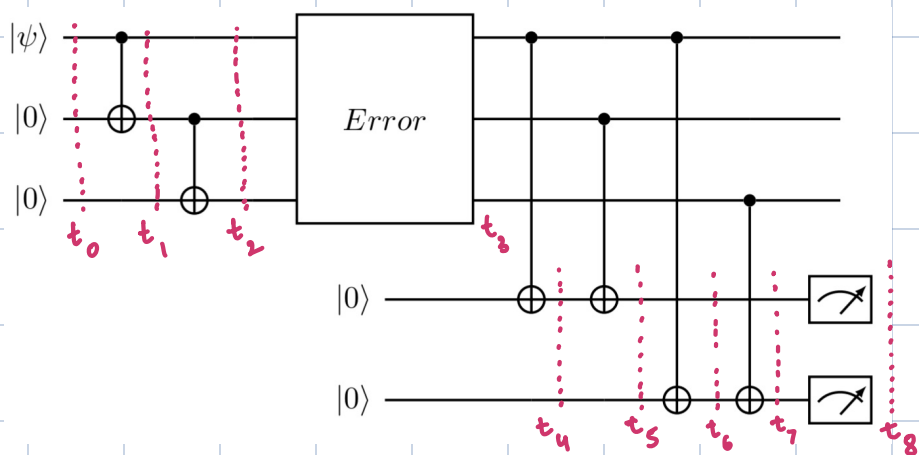


1.a.



$$|\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle, \quad \alpha = \beta = \frac{1}{\sqrt{2}}$$

$$t_0 = (\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle + \beta |1\rangle \otimes |0\rangle \otimes |0\rangle)$$

$$t_1 = (\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle + \beta |1\rangle \otimes |1\rangle \otimes |0\rangle)$$

$$t_2 = (\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle + \beta |1\rangle \otimes |1\rangle \otimes |1\rangle)$$

$$t_3 = (\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle \otimes |00\rangle - \beta |1\rangle \otimes |1\rangle \otimes |1\rangle \otimes |00\rangle)$$

$$t_4 = ([\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle] \otimes |00\rangle - [\beta |1\rangle \otimes |1\rangle \otimes |1\rangle] \otimes |10\rangle)$$

$$t_5 = ([\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle] \otimes |00\rangle - [\beta |1\rangle \otimes |1\rangle \otimes |1\rangle] \otimes |00\rangle)$$

$$t_6 = ([\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle] \otimes |00\rangle - [\beta |1\rangle \otimes |1\rangle \otimes |1\rangle] \otimes |01\rangle)$$

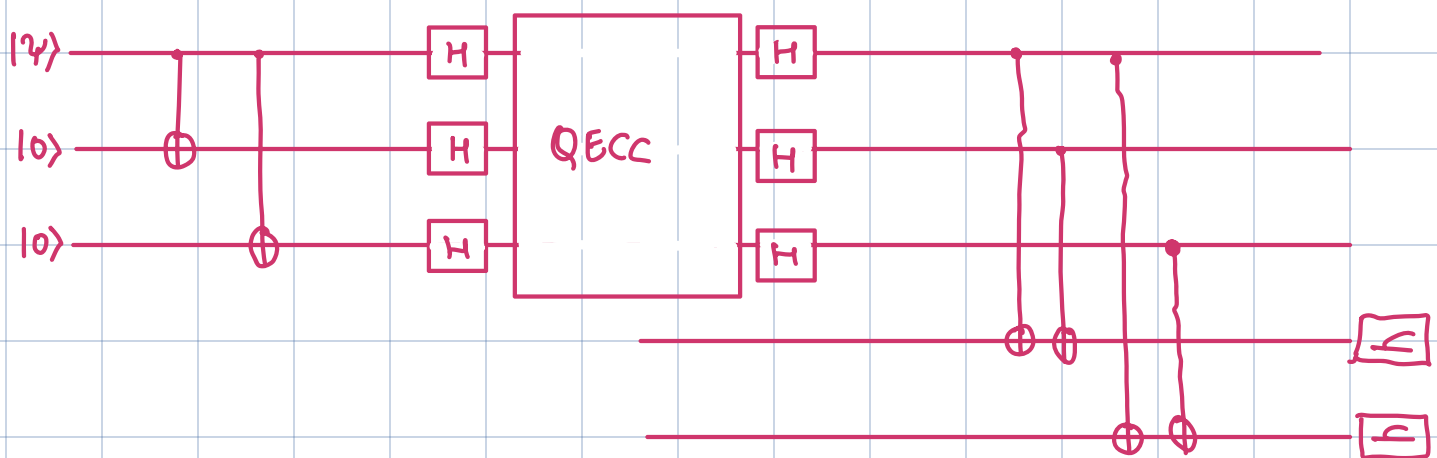
$$t_7 = ((\alpha |0\rangle \otimes |0\rangle \otimes |0\rangle) \otimes |00\rangle - (\beta |1\rangle \otimes |1\rangle \otimes |1\rangle) \otimes |00\rangle)$$

$$t_8 = \left(\frac{1}{\sqrt{2}} (|0\rangle \otimes |0\rangle \otimes |0\rangle) - \frac{1}{\sqrt{2}} (|1\rangle \otimes |1\rangle \otimes |1\rangle) \right)$$

$$= \frac{1}{\sqrt{2}} (|000\rangle - |111\rangle)$$

It will not detect the phase error and will not be able to correct it.

1. b.



2.

```
[20] # @title
!pip install qiskit-terra==0.21.2 qiskit-aer==0.10.4 qiskit-ibmq-provider==0.19.2 qiskit==0.37.2 qiskit
```

```
# initialization
import numpy as np

# importing Qiskit
from qiskit import IBMQ, Aer
from qiskit.providers.ibmq import least_busy
from qiskit import QuantumCircuit, transpile

# import basic plot tools
from qiskit.visualization import plot_histogram

# set the length of the n-bit input string.
n = 3

const_oracle = QuantumCircuit(n+1)

output = np.random.randint(2)
if output == 1:
    const_oracle.x(n)

const_oracle.draw()
```

q_0:

q_1:

q_2:

q_3:

```
[22] balanced_oracle = QuantumCircuit(n+1)
b_str = "101"
```

```
# Place X-gates
for qubit in range(len(b_str)):
    if b_str[qubit] == '1':
        balanced_oracle.x(qubit)
balanced_oracle.draw()
```

q_0: 

q_1: _____

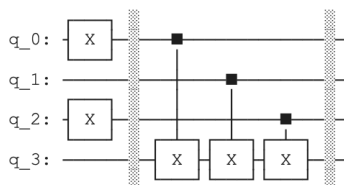
q_2: 

q_3: _____

```
[23] # Use barrier as divider
balanced_oracle.barrier()

# Controlled-NOT gates
for qubit in range(n):
    balanced_oracle.cx(qubit, n)

balanced_oracle.barrier()
balanced_oracle.draw()
```



```
[24] balanced_oracle = QuantumCircuit(n+1)
    b_str = "101"

    # Place X-gates
    for qubit in range(len(b_str)):
        if b_str[qubit] == '1':
            balanced_oracle.x(qubit)

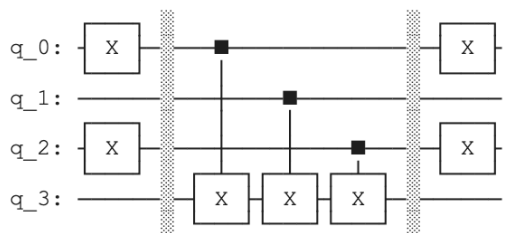
    # Use barrier as divider
    balanced_oracle.barrier()

    # Controlled-NOT gates
    for qubit in range(n):
        balanced_oracle.cx(qubit, n)

    balanced_oracle.barrier()

    # Place X-gates
    for qubit in range(len(b_str)):
        if b_str[qubit] == '1':
            balanced_oracle.x(qubit)

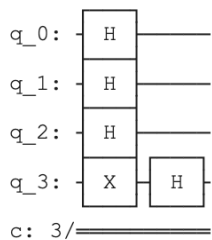
    # Show oracle
    balanced_oracle.draw()
```



```
[25] dj_circuit = QuantumCircuit(n+1, n)

    # Apply H-gates
    for qubit in range(n):
        dj_circuit.h(qubit)

    # Put qubit in state |->
    dj_circuit.x(n)
    dj_circuit.h(n)
    dj_circuit.draw()
```

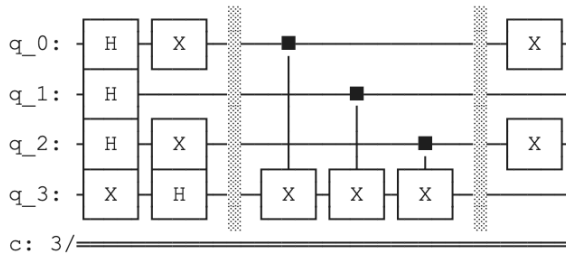


```
[26] dj_circuit = QuantumCircuit(n+1, n)

# Apply H-gates
for qubit in range(n):
    dj_circuit.h(qubit)

# Put qubit in state  $|-\rangle$ 
dj_circuit.x(n)
dj_circuit.h(n)

# Add oracle
dj_circuit = dj_circuit.compose(balanced_oracle)
dj_circuit.draw()
```



```
[27] dj_circuit = QuantumCircuit(n+1, n)

# Apply H-gates
for qubit in range(n):
    dj_circuit.h(qubit)

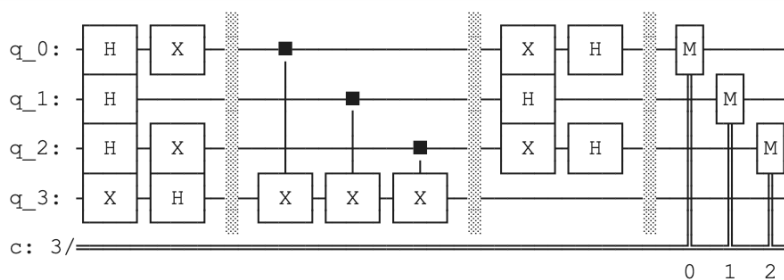
# Put qubit in state  $|-\rangle$ 
dj_circuit.x(n)
dj_circuit.h(n)

# Add oracle
dj_circuit = dj_circuit.compose(balanced_oracle)

# Repeat H-gates
for qubit in range(n):
    dj_circuit.h(qubit)
dj_circuit.barrier()

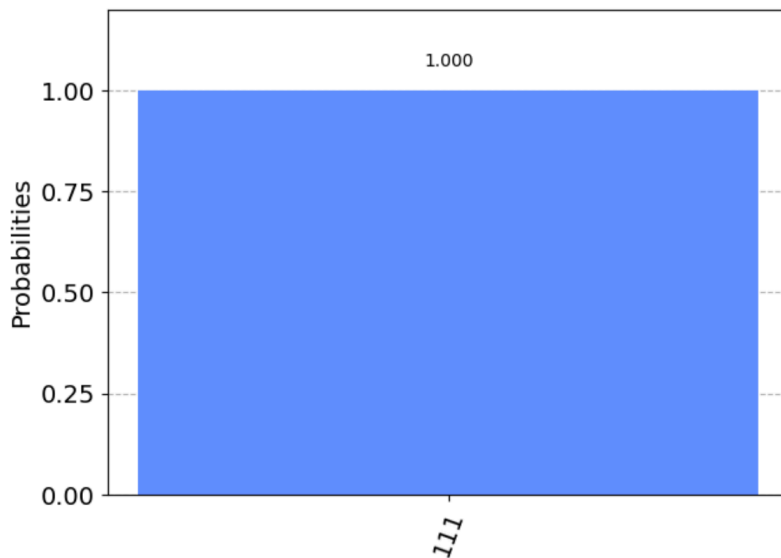
# Measure
for i in range(n):
    dj_circuit.measure(i, i)

# Display circuit
dj_circuit.draw()
```



```
[28] # use local simulator
aer_sim = Aer.get_backend('aer_simulator')
results = aer_sim.run(dj_circuit).result()
answer = results.get_counts()

plot_histogram(answer)
```



```
def dj_oracle(case, n):
    # We need to make a QuantumCircuit object to return
    # This circuit has n+1 qubits: the size of the input,
    # plus one output qubit
    oracle_qc = QuantumCircuit(n+1)

    # First, let's deal with the case in which oracle is balanced
    if case == "balanced":
        # First generate a random number that tells us which CNOTs to
        # wrap in X-gates:
        b = np.random.randint(1,2**n)
        # Next, format 'b' as a binary string of length 'n', padded with zeros:
        b_str = format(b, '0'+str(n)+'b')
        # Next, we place the first X-gates. Each digit in our binary string
        # corresponds to a qubit, if the digit is 0, we do nothing, if it's 1
        # we apply an X-gate to that qubit:
        for qubit in range(len(b_str)):
            if b_str[qubit] == '1':
                oracle_qc.x(qubit)
        # Do the controlled-NOT gates for each qubit, using the output qubit
        # as the target:
        for qubit in range(n):
            oracle_qc.cx(qubit, n)
        # Next, place the final X-gates
        for qubit in range(len(b_str)):
            if b_str[qubit] == '1':
                oracle_qc.x(qubit)

    # Case in which oracle is constant
    if case == "constant":
        # First decide what the fixed output of the oracle will be
        # (either always 0 or always 1)
        output = np.random.randint(2)
        if output == 1:
            oracle_qc.x(n)

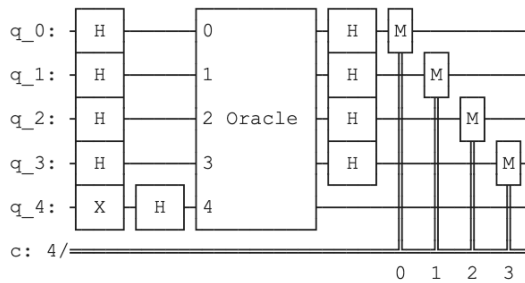
    oracle_gate = oracle_qc.to_gate()
    oracle_gate.name = "Oracle" # To show when we display the circuit
    return oracle_gate
```

```
[30] def dj_algorithm(oracle, n):
    dj_circuit = QuantumCircuit(n+1, n)
    # Set up the output qubit:
    dj_circuit.x(n)
    dj_circuit.h(n)
    # And set up the input register:
    for qubit in range(n):
        dj_circuit.h(qubit)
    # Let's append the oracle gate to our circuit:
    dj_circuit.append(oracle, range(n+1))
    # Finally, perform the H-gates again and measure:
    for qubit in range(n):
        dj_circuit.h(qubit)

    for i in range(n):
        dj_circuit.measure(i, i)

    return dj_circuit

n = 4
oracle_gate = dj_oracle('balanced', n)
dj_circuit = dj_algorithm(oracle_gate, n)
dj_circuit.draw()
```



```
[31] transpiled_dj_circuit = transpile(dj_circuit, aer_sim)
results = aer_sim.run(transpiled_dj_circuit).result()
answer = results.get_counts()
plot_histogram(answer)
```

